

LUCAS HENRIQUE LOPES COSTA

DESENVOLVIMENTO DO FRONT-END DE UMA APLICAÇÃO WEB PARA AGRICULTURA 4.0 NA EMPRESA ZEEWAY

LAVRAS – MG

2026

LUCAS HENRIQUE LOPES COSTA

DESENVOLVIMENTO DO FRONT-END DE UMA APLICAÇÃO WEB PARA AGRICULTURA 4.0 NA EMPRESA ZEEWAY

Relatório técnico apresentada à Universidade Federal de Lavras, como parte das exigências do programa de Graduação em Sistemas de informação, para a obtenção do título de Bacharel em Sistema de Informação

Prof. Paulo Afonso Parreira Júnior
Orientador

LAVRAS – MG
2026

Ficha catalográfica elaborada pela Coordenadoria de Processos Técnicos da Biblioteca Universitária da UFLA, com dados informados pelo(a) próprio(a) autor(a).

Costa, Lucas Henrique Lopes
Desenvolvimento do Front-end de uma Aplicação Web para Agricultura 4.0 na Empresa Zeeway / Lucas Henrique Lopes Costa. – Lavras : UFLA, 2026.
41p. : il.

Trabalho de Conclusão de Curso (graduação)–Universidade Federal de Lavras, 2026.
Orientador: Prof. Paulo Afonso Parreira Júnior.
Bibliografia.

1. Desenvolvimento web. 2. React. 3. TypeScript. 4. Scrum. I. Universidade Federal de Lavras. II. Título.

CDD [Número de Classificação]

LUCAS HENRIQUE LOPES COSTA

DESENVOLVIMENTO DO FRONT-END DE UMA APLICAÇÃO WEB PARA AGRICULTURA 4.0 NA EMPRESA ZEEWAY

Relatório técnico apresentada à Universidade Federal de Lavras, como parte das exigências do programa de Graduação em Sistemas de informação, para a obtenção do título de Bacharel em Sistema de Informação

APROVADA em _____ de _____ de _____.

Prof. Paulo Afonso Parreira Júnior
Orientador

LAVRAS – MG
2026

Dedico este trabalho a Deus, que guiou os meus passos. Aos meus pais, Lidiana e Antônio, ao meu irmão Marcos Túlio, e a toda a minha família, que sempre me deram apoio para chegar onde estou.

AGRADECIMENTOS

Agradeço primeiramente a Deus, por me dar saúde, resiliência e por guiar os meus passos ao longo de toda esta jornada universitária.

Aos meus pais, Lidiana e Antônio, e ao meu irmão, Marcos Túlio, o meu muito obrigado por serem a minha base. Gratidão pelo apoio incondicional de vocês.

Um agradecimento muito especial aos meus avós, Maria Aparecida e Waldeli; e meus padrinhos, Alessandra e Hélio, por todo o incentivo. Às minhas primas de coração, Fernanda, Júlia e Laura, agradeço por estarem sempre ao meu lado, me acompanhando e se preocupando comigo em cada etapa desse processo. Ao meus tios, Kelson, Ana, Kerley, por também estarem comigo nessa trajetória.

Gostaria de expressar minha profunda gratidão ao Luiz Fernando e ao seu pai, José Ari Ferreira. Vocês foram as pessoas que me motivaram a vir para a UFLA, que me deram a primeira grande oportunidade de trabalho e que acreditaram em mim. Tenho um enorme carinho e uma gratidão imensa por tudo o que construímos e vivemos juntos na Zeeway.

Por fim, agradeço ao professor Paulo Afonso pela orientação, paciência e por compartilhar sua experiência para a construção deste trabalho, e a todos os professores do curso que contribuíram para a minha formação profissional e pessoal.

RESUMO

Este trabalho apresenta as atividades desenvolvidas durante o estágio supervisionado na empresa Zeeway, sediada em Lavras (MG), no desenvolvimento de uma plataforma web sob demanda para uma empresa de pesquisa agrícola de Lavras, atuante no agronegócio e especializada em análises de solos e sementes. A atuação concentrou-se na construção da camada de interface (*front-end*) da plataforma, responsável pelo registro e pela análise dos protocolos de pesquisa, em substituição ao uso descentralizado de planilhas. As tecnologias adotadas foram o *React* e o *TypeScript*, com componentes do *Material UI*, formulários com *Formik* e *Yup*, gerenciamento de estado global com *Zustand*, comunicação com a API por meio de *Axios* e *React Query*, e visualização de dados com *ApexCharts*. O desenvolvimento foi conduzido pela metodologia ágil *Scrum*, com *Sprints* quinzenais. As atividades incluíram a autenticação com *JSON Web Tokens* (JWT) e a renovação automática do *token*, o controle de acesso por perfil no nível das rotas, formulários extensos com validação declarativa, a geração automatizada de gráficos e *dashboards* e a exportação de imagens com a identidade visual do cliente. A experiência consolidou competências em desenvolvimento web e em visualização de dados, e permitiu vivenciar a engenharia de *software* em equipe, com cliente externo e regras de negócio reais.

Palavras-chave: desenvolvimento web; React; TypeScript; Scrum; agronegócio.

ABSTRACT

This report presents the activities developed during the supervised internship at Zeeway, a company based in Lavras (MG), in the development of a custom web platform for an agricultural research company also based in Lavras, working in the agribusiness sector and specialized in soil and seed analysis. The work focused on building the front-end layer of the platform, responsible for registering and analyzing research protocols, replacing the decentralized use of spreadsheets. The technologies adopted were React and TypeScript, with Material UI components, forms with Formik and Yup, global state management with Zustand, API communication through Axios and React Query, and data visualization with ApexCharts. The development followed the Scrum agile methodology, with biweekly Sprints. The activities included authentication with JSON Web Tokens (JWT) and automatic token refresh, role-based access control at the route level, extensive forms with declarative validation, automated generation of charts and dashboards, and image export with the client's visual identity. The experience consolidated competencies in web development and data visualization, and provided practical software engineering experience in a team, with an external client and real business rules.

Keywords: front-end development; React; TypeScript; Agriculture 4.0; agribusiness.

LISTA DE FIGURAS

Figura 2.1 – Exemplo de aplicação construída com o *Material UI*. 16
Figura 2.2 – Renderização aproximada do gráfico de barras configurado no Código 2.3. . 17

Figura 3.1 – Tela de *backlog* do *Azure DevOps Boards*. 20
Figura 3.2 – Arquitetura em camadas do *front-end* da plataforma de protocolos. 21

Figura 4.1 – Tela de *login* da plataforma. 26
Figura 4.2 – Tela de gestão de usuários. 27
Figura 4.3 – Modais de filtro, adição, exclusão e *feedback* da gestão de usuários. 28
Figura 4.4 – Aba de Avaliações do formulário de protocolo. 30
Figura 4.5 – Tela de Informações do módulo de orçamentos e componentes reutilizados. 31
Figura 4.6 – Fluxo de geração de gráficos a partir dos dados de avaliação. 33
Figura 4.7 – Gráficos de barras gerados automaticamente para uma avaliação. 35
Figura 4.8 – Imagem PNG exportada de um gráfico, com a identidade visual da empresa. 36
Figura 4.9 – *Dashboard* geral com a visão consolidada dos protocolos. 37

SUMÁRIO

1 INTRODUÇÃO 9
1.1 Contextualização 9
1.2 A Empresa Cliente e o Cenário do Projeto 9
1.3 Objetivos do Estágio e do TCC 10
2 FUNDAMENTAÇÃO TEÓRICA 11
2.1 Gestão de Dados no Agronegócio 11
2.2 Metodologias Ágeis: O *Framework Scrum* 12
2.3 Tecnologias de Desenvolvimento *Front-end* 12
2.3.1 React e TypeScript 13
2.3.2 Gerenciamento de Estado e Formulários 14
2.3.3 Design de Interface com Material UI (MUI) 15
2.3.4 Visualização de Dados com ApexCharts 16
3 METODOLOGIA E PROCESSO DE DESENVOLVIMENTO 19
3.1 Aplicação do *Scrum* no Projeto 19
3.2 Levantamento de Requisitos e Regras de Negócio 20
3.3 Arquitetura do *Front-end* e Estrutura do Projeto 20
4 RESULTADOS: A PLATAFORMA DE PROTOCOLOS 24
4.1 Módulo de Autenticação e Controle de Acesso 24
4.2 Módulo de Registros Agrícolas 27
4.2.1 Listagem de Protocolos 27
4.2.2 Formulário de Cadastro e Edição de Protocolos 28
4.3 Módulo de Análise e Dashboards 32
4.3.1 Geração Automatizada de Gráficos 32
4.3.2 Dashboard Geral 35
5 CONSIDERAÇÕES FINAIS 38
5.1 Avaliação do Produto Desenvolvido 38
5.2 Impacto do Estágio na Formação Acadêmica e Profissional 39
REFERÊNCIAS 41

1 INTRODUÇÃO

Neste capítulo introdutório, serão apresentados o contexto do projeto desenvolvido durante o estágio supervisionado, a empresa cliente e o cenário do agronegócio que motivou a construção da plataforma de protocolos. Serão também delineados os objetivos do estágio e deste relatório, que visa documentar as atividades realizadas, as tecnologias adotadas e os resultados alcançados.

1.1 Contextualização

O agronegócio é um dos setores mais importantes e dinâmicos da economia brasileira, com participação relevante no Produto Interno Bruto e forte impacto sobre emprego, renda e exportações (Centro de Estudos Avançados em Economia Aplicada; Confederação da Agricultura e Pecuária do Brasil, 2025). Dentro desse cenário, a pesquisa agrícola focada na análise de solos e sementes desempenha papel central para elevar a produtividade e sustentar decisões técnicas no campo. Com o avanço da chamada Agricultura 4.0, a coleta, o armazenamento e a análise de dados tornaram-se indispensáveis para a tomada de decisões mais rápidas e precisas (Empresa Brasileira de Pesquisa Agropecuária, 2019).

Apesar da crescente digitalização, muitas instituições de pesquisa ainda enfrentam o desafio da fragmentação de dados, dependendo de ferramentas manuais e planilhas eletrônicas para registrar suas descobertas. Esse cenário dificulta a rastreabilidade das informações, a aplicação de regras de negócio complexas e, principalmente, a geração de análises visuais rápidas que poderiam acelerar os resultados das pesquisas (Empresa Brasileira de Pesquisa Agropecuária, 2019). É nesse contexto de modernização e centralização de dados que se insere o projeto desenvolvido durante o estágio.

1.2 A Empresa Cliente e o Cenário do Projeto

As atividades de estágio supervisionado foram realizadas na empresa Zeeway, empresa lavrense de tecnologia fundada no ecossistema de inovação da Universidade Federal de Lavras (UFLA) (Zeeway Soluções Digitais, 2026). A Zeeway atua no desenvolvimento de *software* e na entrega de soluções tecnológicas personalizadas, destacando-se pelo compromisso com a

qualidade dos projetos e com os resultados de seus clientes (Zeeway Soluções Digitais, 2026). Atualmente, a empresa conta com um quadro de 15 a 30 colaboradores diretos e mantém uma base aproximada de 40 clientes diretos e 500 clientes indiretos. O trabalho principal desenvolvido durante o período de estágio foi a construção de uma plataforma web encomendada por uma empresa de pesquisa agrícola sediada em Lavras (MG), atuante no setor do agronegócio e especializada em análises de solos e sementes.

A necessidade dessa empresa cliente consistia em abandonar o uso descentralizado de planilhas e adotar uma plataforma própria para registrar os seus protocolos, que são registros detalhados com as informações de safra, componentes do solo e produtividade de grãos. O acesso ao sistema seria distribuído aos colaboradores, exigindo uma arquitetura robusta para lidar com diferentes perfis de usuário e permissões de acesso.

A participação no projeto concentrou-se no desenvolvimento da interface visual e da experiência do usuário (*front-end*). A aplicação foi construída utilizando a biblioteca React com TypeScript, empregando componentes da interface Material UI (MUI) e integrando a biblioteca ApexCharts para o diferencial do sistema: a geração automatizada de gráficos de análise agronômica. Todo o ciclo de desenvolvimento foi pautado pela metodologia ágil Scrum, permitindo organização e entregas incrementais.

1.3 Objetivos do Estágio e do TCC

O objetivo principal do estágio foi aplicar, em um ambiente corporativo real, os conhecimentos adquiridos ao longo da graduação, vivenciando a rotina de desenvolvimento de *software* em equipe e os desafios da engenharia de requisitos.

Este relatório tem como objetivo apresentar de forma estruturada as atividades desempenhadas na Zeeway, detalhando a arquitetura *front-end* construída, as tecnologias adotadas e a metodologia de trabalho. Busca-se demonstrar também os desafios técnicos enfrentados na tradução de regras de negócio complexas para a interface da plataforma e o impacto da experiência na formação profissional e acadêmica.

2 FUNDAMENTAÇÃO TEÓRICA

A atividade de desenvolvimento de *software* envolve o uso de linguagens de programação, bibliotecas, *frameworks* e metodologias de trabalho para produzir, testar e entregar sistemas. Neste capítulo são apresentados os conceitos e tecnologias que fundamentaram as atividades realizadas durante o estágio, abrangendo o contexto do agronegócio, a metodologia ágil *Scrum* e as tecnologias de desenvolvimento *front-end* utilizadas na construção da plataforma.

2.1 Gestão de Dados no Agronegócio

O agronegócio brasileiro é responsável por uma parcela significativa do Produto Interno Bruto nacional. De acordo com dados do Centro de Estudos Avançados em Economia Aplicada (Cepea), em parceria com a Confederação da Agricultura e Pecuária do Brasil (CNA), o setor tem apresentado crescimento consistente nas últimas décadas, consolidando o país como um dos maiores produtores e exportadores de alimentos do mundo (Centro de Estudos Avançados em Economia Aplicada; Confederação da Agricultura e Pecuária do Brasil, 2025).

Nesse contexto, a pesquisa agrícola desempenha papel fundamental para a manutenção e o aumento da produtividade. Instituições e empresas especializadas em análise de solos e sementes coletam grandes volumes de dados ao longo das safras, incluindo informações sobre composição do solo, tratamentos aplicados, condições climáticas e resultados de produtividade. Tradicionalmente, esses dados são registrados em planilhas eletrônicas e documentos avulsos, o que dificulta a rastreabilidade, a padronização e a análise integrada das informações (Empresa Brasileira de Pesquisa Agropecuária, 2019).

Com o avanço da chamada Agricultura 4.0, a digitalização dos processos agrícolas tornou-se uma tendência crescente. A Agricultura 4.0 é caracterizada pela adoção de tecnologias digitais, como sensores, *Internet das Coisas* (IoT), computação em nuvem e análise de dados, com o objetivo de otimizar a tomada de decisões no campo (Massruhá; Leite *et al.*, 2020). Nesse cenário, plataformas *web* que centralizam o registro e a análise de dados de pesquisa representam uma evolução em relação ao uso descentralizado de planilhas, permitindo que os colaboradores acessem informações atualizadas, apliquem regras de negócio padronizadas e gerem visualizações gráficas de forma automatizada.

2.2 Metodologias Ágeis: O *Framework Scrum*

O *Scrum* é um *framework* de gerenciamento de projetos e desenvolvimento ágil com formato iterativo e incremental. Cada ciclo no *Scrum* é conhecido como *Sprint*, um processo com duração de tempo fixa e com um conjunto de tarefas que devem ser implementadas nesse período. No final de cada *Sprint*, uma nova versão com novas funcionalidades e melhorias do sistema é entregue (Schwaber; Sutherland, 2020).

No *Scrum*, antes do início de uma *Sprint*, ocorre uma reunião de planejamento chamada *planning*. Nela, o *Product Owner* seleciona as tarefas do *Product Backlog*, que é a lista priorizada com todas as funcionalidades e melhorias planejadas para o produto, e as adiciona ao *Sprint Backlog*, definindo o escopo de trabalho da *Sprint*. O *Product Owner* é o responsável por definir a prioridade das tarefas e planejar o lançamento de novas funcionalidades.

Outra figura-chave no processo do *Scrum* é o *Scrum Master*, pessoa responsável por garantir que os conceitos da metodologia estão sendo aplicados de maneira correta e por remover impedimentos que possam atrapalhar o progresso da equipe. A equipe de desenvolvimento, por sua vez, é composta pelos profissionais que efetivamente executam as tarefas planejadas.

Além da *planning*, o *Scrum* prevê reuniões diárias chamadas *daily meetings*, nas quais cada membro da equipe compartilha o que foi feito no dia anterior, o que será feito no dia atual e se há algum impedimento. No final da *Sprint*, ocorrem a *Sprint Review*, na qual o incremento do produto é apresentado, e a *Sprint Retrospective*, na qual a equipe discute o que funcionou bem e o que pode ser melhorado no processo.

No projeto desenvolvido durante o estágio na Zeeway, o *Scrum* foi adotado como metodologia de desenvolvimento, com *Sprints* de duração fixa, reuniões de planejamento e acompanhamento por meio de *dailies*. A adoção dessa metodologia permitiu entregas incrementais ao cliente e uma organização clara das tarefas ao longo do projeto.

2.3 Tecnologias de Desenvolvimento *Front-end*

Nesta seção são apresentadas as principais tecnologias utilizadas no desenvolvimento da interface da plataforma de protocolos. A escolha dessas tecnologias levou em consideração a necessidade de construir uma aplicação *web* interativa, com formulários complexos, tabelas dinâmicas e visualizações gráficas.

2.3.1 React e TypeScript

O *React* é uma biblioteca *JavaScript* de código aberto, criada pelo *Facebook* (atualmente *Meta*), voltada para a construção de interfaces de usuário. Seu principal conceito é a componentização: a interface é dividida em componentes reutilizáveis, cada um responsável por renderizar uma parte específica da tela. Os componentes podem receber dados por meio de propriedades (*props*) e gerenciar seu próprio estado interno utilizando *hooks* como `useState` e `useEffect` (Meta Platforms, 2024).

Uma das vantagens do *React* é o uso do *Virtual DOM* (*Document Object Model*), uma representação virtual da árvore de elementos da página. Quando o estado de um componente é alterado, o *React* compara a versão atual do *Virtual DOM* com a anterior e atualiza apenas os elementos que de fato mudaram, otimizando a performance de renderização.

O *TypeScript* é um superconjunto do *JavaScript* desenvolvido pela *Microsoft* que adiciona tipagem estática à linguagem. Em projetos *React*, o *TypeScript* permite definir tipos para as *props* dos componentes, para os dados recebidos de APIs e para o estado da aplicação, reduzindo erros em tempo de desenvolvimento e facilitando a manutenção do código (Microsoft, 2024). O Código 2.1 apresenta um exemplo didático de um componente React escrito em TypeScript.

Código 2.1 – Exemplo didático de um componente *React* com *TypeScript*.

```tsx
import { useState } from "react";

interface ContadorProps {
  inicial: number;
  passo?: number;
}

export function Contador({ inicial, passo = 1 }: ContadorProps) {
  const [valor, setValor] = useState<number>(inicial);

  return (
    <button onClick={() => setValor(valor + passo)}>
      Cliques: {valor}
    </button>
  );
}
```

No exemplo, a interface `ContadorProps` declara as propriedades aceitas pelo componente, indicando que `inicial` é obrigatória e que `passo` é opcional (sinalizada pela interrogação) com valor padrão igual a um. O componente `Contador` recebe essas propriedades já tipadas, mantém um estado interno por meio do *hook* `useState` (cuja tipagem também é explícita) e retorna um elemento de interface (botão) cujo clique incrementa o valor do estado, fazendo com que o *React* reexecute a função e atualize a tela com o novo valor.

No projeto da plataforma de protocolos, a combinação de *React* com *TypeScript* foi utilizada para construir todos os componentes da interface. A ferramenta de *build* adotada foi o *Vite*, que oferece um servidor de desenvolvimento com *Hot Module Replacement* (HMR), permitindo que as alterações no código fossem refletidas instantaneamente no navegador durante o desenvolvimento.

2.3.2 Gerenciamento de Estado e Formulários

Em aplicações *React* de maior porte, o gerenciamento do estado da aplicação torna-se um desafio. Para solucionar esse problema, foi utilizada a biblioteca *Zustand*, uma solução leve de gerenciamento de estado global que permite compartilhar dados entre componentes sem a necessidade de passar *props* por múltiplos níveis da árvore de componentes (Kato, 2024). Em conjunto com o *Zustand*, foram empregados o seu *middleware* de persistência, para manter a sessão do usuário entre acessos, e a biblioteca *Immer*, que permite atualizar o estado de forma imutável por meio de uma sintaxe direta. A biblioteca *jwt-decode* foi utilizada para decodificar o *token* JWT recebido no login e extrair as informações do usuário.

Para o gerenciamento dos formulários, que constituíam uma parte central da plataforma dado o grande número de campos nos protocolos, foram utilizadas as bibliotecas *Formik* e *Yup*. O *Formik* é uma biblioteca que simplifica a criação de formulários em *React*, cuidando do estado dos campos, da validação e do envio dos dados. O *Yup* é uma biblioteca de validação de esquemas que permite definir regras de validação de forma declarativa, como campos obrigatórios, tipos de dados esperados e validações condicionais (Palmer, 2024). O Código 2.2 apresenta um exemplo didático de um esquema de validação com *Yup*.

Código 2.2 – Exemplo didático de um esquema de validação com *Yup*.

```
1 import * as Yup from "yup";
2
```

```
const esquemaUsuario = Yup.object().shape({
  nome: Yup.string().required("Nome obrigatorio"),
  email: Yup.string().email("E-mail invalido").required("E-mail
      obrigatorio"),
  idade: Yup.number().min(18, "Idade minima 18").nullable(),
});
```

No exemplo, `Yup.object().shape` declara o formato esperado de um objeto. Cada chave do objeto encadeia validações específicas: `nome` deve ser uma *string* obrigatória; `email` deve ter formato válido e ser obrigatório, combinando duas regras na mesma cadeia; `idade` deve ser um número de pelo menos dezoito, mas pode receber valor nulo. A mensagem passada a cada validação é o texto exibido ao usuário quando a regra é violada.

A comunicação com a API do servidor foi implementada por meio da biblioteca *Axios*, um cliente HTTP que permite realizar requisições de forma simplificada. Para o gerenciamento de cache e sincronização dos dados do servidor, foi utilizada a biblioteca *React Query* (*TanStack Query*), que automatiza o processo de busca, cache, atualização e invalidação de dados, evitando requisições desnecessárias e mantendo a interface atualizada (Linsley, 2024).

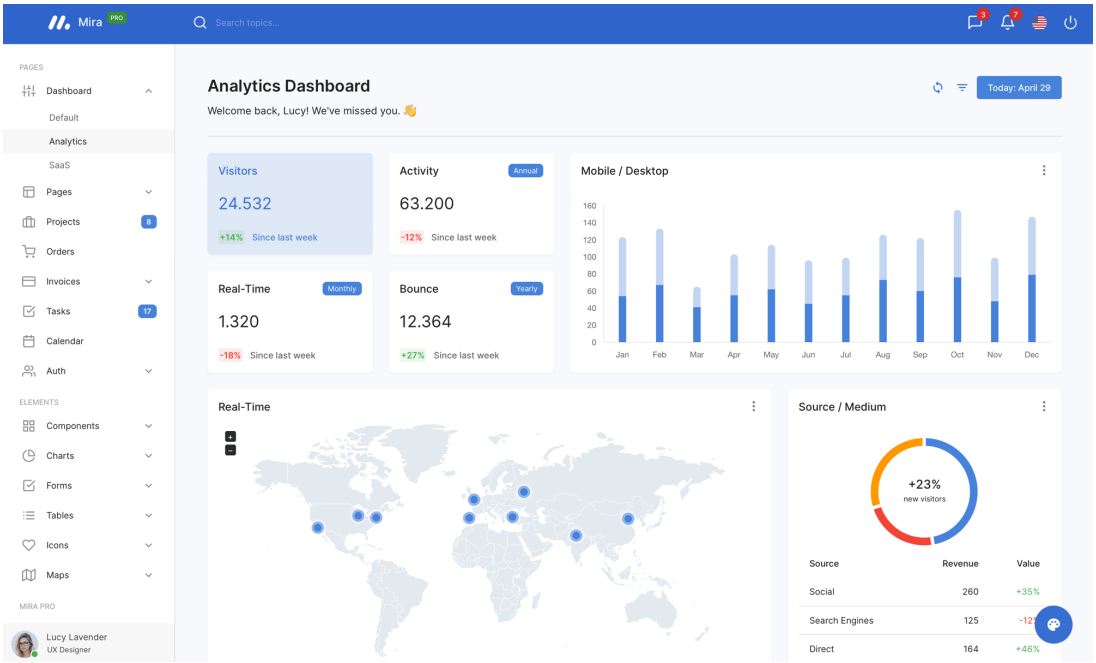
2.3.3 Design de Interface com Material UI (MUI)

O *Material UI* (MUI) é uma biblioteca de componentes *React* que implementa o *Material Design*, sistema de design criado pelo *Google*. A biblioteca fornece um conjunto amplo de componentes prontos, como botões, campos de texto, tabelas, menus, modais e *date pickers*, todos com aparência e comportamento consistentes (MUI, 2024). A Figura 2.1 apresenta um exemplo de aplicação construída com o *Material UI*, divulgado pela própria biblioteca, em que diferentes componentes (menu lateral, *cards* de métricas, gráficos e tabelas) são combinados de forma coesa.

Uma das vantagens do *MUI* é a possibilidade de personalização por meio de temas. No projeto desenvolvido durante o estágio, foi criado um tema personalizado que definia as cores, tipografia e espaçamentos de acordo com a identidade visual da empresa cliente. Dessa forma, todos os componentes da biblioteca adotavam automaticamente o padrão visual definido, garantindo consistência em toda a plataforma.

Além dos componentes básicos do *MUI*, o projeto utilizou a biblioteca *MUI X Date Pickers* para os campos de seleção de data, recurso frequentemente utilizado nos formulários

Figura 2.1 – Exemplo de aplicação construída com o *Material UI*.



Fonte: Material UI. Disponível em: https://mui.com/. Acesso em: maio 2026.

de protocolos que exigiam o registro de datas de plantio, colheita e emergência. Também foi utilizada a biblioteca *TanStack React Table* para a construção de tabelas com funcionalidades avançadas de ordenação, paginação e filtragem.

2.3.4 Visualização de Dados com ApexCharts

A geração de gráficos e visualizações de dados constituiu o principal diferencial da plataforma em relação ao uso de planilhas. Para essa funcionalidade, foi utilizada a biblioteca *ApexCharts*, uma biblioteca *JavaScript* de código aberto para a criação de gráficos interativos. A integração com o *React* foi realizada por meio do pacote *react-apexcharts* (Chhipa, 2024).

O *ApexCharts* oferece diversos tipos de gráficos, como barras, linhas, pizza, radar e área, além de funcionalidades como *zoom*, *tooltip* e exportação em formato de imagem. Na plataforma de protocolos, foram utilizados principalmente gráficos de barras verticais e horizontais para comparar os resultados entre diferentes tratamentos e momentos de avaliação. O Código 2.3 apresenta um exemplo didático da configuração mínima de um gráfico de barras com o pacote *react-apexcharts*.

Código 2.3 – Exemplo didático de gráfico de barras com *react-apexcharts*.

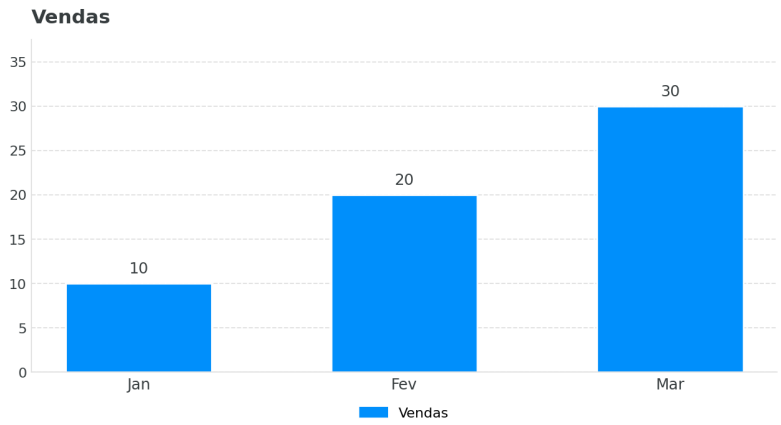
```
1 import ApexCharts from "react-apexcharts";
```

```
2
3  const series = [{ name: "Vendas", data: [10, 20, 30] }];
4  const options = {
5    chart: { type: "bar" },
6    xaxis: { categories: ["Jan", "Fev", "Mar"] },
7  };
8
9  export function GraficoVendas() {
10   return <ApexCharts options={options} series={series} type="
       bar" />;
11 }
```

No exemplo, `series` é a lista de séries do gráfico, cada uma contendo um nome e um vetor de valores numéricos correspondentes a cada categoria. O objeto `options` concentra a configuração visual, definindo o tipo de gráfico (`chart.type`) e os rótulos do eixo horizontal (`xaxis.categories`). O componente `ApexCharts` recebe esses dois objetos e cuida da renderização do gráfico na tela.

A Figura 2.2 apresenta uma representação aproximada do gráfico produzido pelo Código 2.3, em que as três categorias (Jan, Fev e Mar) são exibidas no eixo horizontal e os valores numéricos correspondentes (10, 20 e 30) compõem as barras verticais.

Figura 2.2 – Renderização aproximada do gráfico de barras configurado no Código 2.3.



Fonte: do autor (2026), com base na biblioteca *ApexCharts*.

Para a avaliação de campos definidos como fórmulas nos protocolos, isto é, expressões matemáticas calculadas a partir dos valores de outros campos, foi utilizada a biblioteca *mathjs*, que interpreta e avalia expressões em tempo de execução. A exportação dos gráficos em formato de imagem, com a identidade visual da empresa cliente, foi implementada com a biblioteca *use-react-screenshot*, que captura um elemento da interface como imagem. Já a biblioteca *JSZip* foi

empregada para empacotar as fotografias de campo de uma avaliação em um único arquivo compactado para *download*.

3 METODOLOGIA E PROCESSO DE DESENVOLVIMENTO

Neste capítulo é descrito o processo de desenvolvimento adotado ao longo do estágio, abrangendo a aplicação da metodologia ágil *Scrum*, o levantamento de requisitos e regras de negócio junto à empresa cliente e a arquitetura do *front-end* construído. Para ilustrar as soluções tecnológicas, são apresentados, ao longo do texto, diagramas de alto nível e trechos simplificados do código desenvolvido, com a remoção de detalhes de estilo e de tratamento de erros para favorecer a legibilidade.

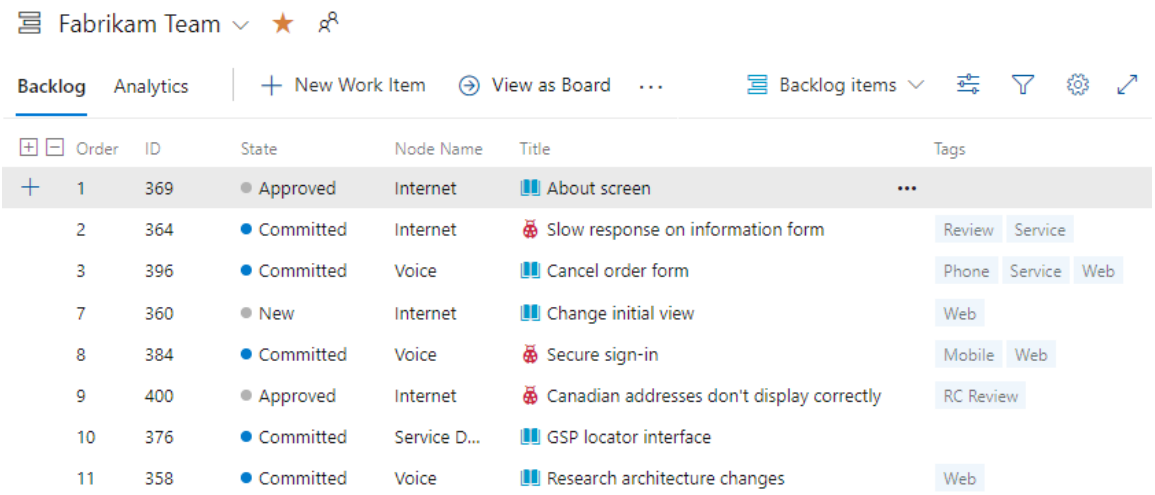
3.1 Aplicação do *Scrum* no Projeto

O desenvolvimento da plataforma seguiu a metodologia *Scrum*, organizada em *Sprints* de duração fixa de duas semanas. No início de cada *Sprint*, era realizada a reunião de *planning*, na qual as tarefas priorizadas do *Product Backlog* eram selecionadas e detalhadas no *Sprint Backlog*. As tarefas eram acompanhadas em um quadro de tarefas dividido nas colunas de afazeres, em andamento, em revisão e concluído.

Para o gerenciamento do *Product Backlog* e do *Sprint Backlog* foi utilizado o *Azure DevOps Boards*, ferramenta da *Microsoft* voltada à gestão ágil de trabalho e ao acompanhamento de *itens de trabalho* (*Work Items*) ao longo do ciclo de desenvolvimento. Cada tarefa do *backlog* possuía identificador, estado (por exemplo, *New*, *Approved* ou *Committed*), título, área de produto e marcadores associados, o que facilitava a priorização e a distribuição das atividades entre os membros da equipe. A Figura 3.1 apresenta um exemplo de tela do *Azure DevOps Boards* com a listagem de *Work Items* de um *backlog*.

O acompanhamento diário ocorria por meio das *daily meetings*, reuniões breves em que cada membro relatava o progresso, os planos do dia e eventuais impedimentos. Ao final de cada *Sprint*, a *Sprint Review* apresentava o incremento desenvolvido ao cliente, e a *Sprint Retrospective* permitia à equipe refletir sobre o processo. A adoção desse fluxo possibilitou entregas incrementais à empresa cliente e a validação contínua das funcionalidades, reduzindo o risco de retrabalho.

Figura 3.1 – Tela de *backlog* do *Azure DevOps Boards*.



Fonte: Microsoft Learn. Disponível em: https://learn.microsoft.com/pt-br/azure/devops/boards/backlogs/create-your-backlog. Acesso em: maio 2026.

3.2 Levantamento de Requisitos e Regras de Negócio

O levantamento de requisitos foi conduzido de forma iterativa, em conjunto com a equipe da empresa cliente. Como a empresa atua na pesquisa agrícola, muitas das regras de negócio eram específicas do domínio e não estavam formalmente documentadas, o que exigiu reuniões frequentes para esclarecer o comportamento esperado da plataforma.

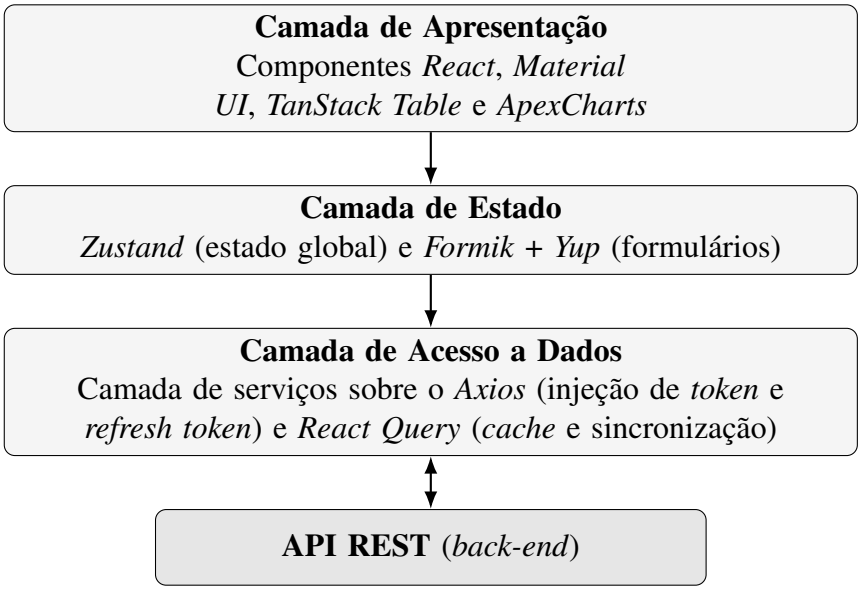
O conjunto de regras que concentrou a maior complexidade foi o das regras de preenchimento dos protocolos, em que a obrigatoriedade e o formato de diversos campos dependiam do tipo de protocolo, da cultura selecionada e da classe do experimento. Como essas combinações não estavam descritas em nenhum documento, cada variação precisava ser esclarecida diretamente com a equipe do cliente antes de ser traduzida para o esquema de validação do formulário. Esses requisitos foram refinados a cada *Sprint*, à medida que as telas eram apresentadas e validadas pelo cliente.

3.3 Arquitetura do *Front-end* e Estrutura do Projeto

A aplicação foi construída como uma *Single Page Application* (SPA) em *React* com *TypeScript*, empacotada pelo *Vite*. A organização do código seguiu uma separação por responsabilidades, distribuída em três camadas lógicas: a camada de apresentação, responsável pela

renderização da interface; a camada de estado, responsável por manter os dados em memória durante a navegação; e a camada de acesso a dados, responsável pela comunicação com a API do servidor. A Figura 3.2 apresenta uma visão de alto nível dessa organização.

Figura 3.2 – Arquitetura em camadas do *front-end* da plataforma de protocolos.



Fonte: do autor (2026).

É importante destacar que o *back-end* da plataforma, incluindo a API REST representada na Figura 3.2, ficou a cargo de outra equipe da Zeeway, dedicada exclusivamente ao desenvolvimento do servidor, da modelagem do banco de dados e da implementação das regras de negócio do lado do servidor. Os capítulos seguintes detalham apenas as atividades realizadas na camada de *front-end*.

A coordenação entre as duas equipes foi baseada em uma estratégia de paralelização do trabalho a partir de uma interface comum. No início de cada nova funcionalidade, eram definidos em conjunto os contratos das chamadas à API, isto é, o caminho de cada *endpoint*, o método HTTP utilizado e a estrutura dos dados de entrada e de saída. Com esse contrato acordado, as duas equipes passavam a trabalhar simultaneamente na mesma *Sprint*: a equipe de *back-end* implementava o *endpoint* correspondente, enquanto a equipe de *front-end* construía a tela e consumia inicialmente os dados em uma versão mascarada (*mock*) que respeitava o contrato. Quando o *endpoint* ficava disponível, a integração era feita substituindo a fonte dos dados pela API real. Ajustes pontuais no contrato eram negociados nas reuniões diárias ou

na *planning* da *Sprint* seguinte, evitando que uma equipe ficasse permanentemente em ritmo diferente da outra.

A estrutura de diretórios do projeto refletia essa separação de responsabilidades, agrupando os arquivos por função, conforme ilustrado no Código 3.1. Os diretórios `components/`, `pages/` e `layouts/` concentram a parte visual da aplicação, enquanto `store/`, `services/` e `routes/` tratam, respectivamente, do estado global, da comunicação com a API e da definição das rotas. O diretório `hooks/` centraliza os *hooks* customizados, como `useForm` e `useRoles`, e `utils/` reúne funções auxiliares reutilizadas em diferentes contextos.

Código 3.1 – Organização simplificada dos diretórios do projeto.

```
src/
  components/   Componentes reutilizaveis (tabelas, modais,
     graficos)
  config/       Constantes e configuracoes globais
  contexts/     Contextos de React
  hooks/        Hooks customizados (useForm, useRoles, React
     Query)
  layouts/      Layouts de pagina (autenticacao e listagem)
  pages/        Telas (Login, Protocols, Dashboard, Clients,
     ...)
  routes/       Definicao e guarda de rotas
  services/     Camada de servicos e chamadas a API (Axios)
  store/        Estado global (Zustand)
  theme/        Tema do Material UI
  types/        Tipos TypeScript compartilhados
  utils/        Funcoes auxiliares (axios, arquivos, debounce)
```

Na camada de acesso a dados, todas as chamadas à API passavam por uma função de serviço (`httpRequest`) que anexava automaticamente o *token* de acesso do usuário autenticado ao cabeçalho de autorização, evitando a repetição dessa lógica em cada requisição. Além disso, foi configurado um interceptador de resposta do *Axios* responsável por renovar o *token* de forma transparente sempre que a API retornava o código de erro 401, reenviando a requisição original com o novo *token* obtido a partir do *refresh token*. O Código 3.2 apresenta um trecho simplificado dessa configuração.

Código 3.2 – Trecho do interceptador de resposta para renovação do *token* (*refresh token*).

```
1 axiosInstance.interceptors.response.use(
2   (response) => response,
3   async (error) => {
4     const originalRequest = error.config;
```

```
5     if (error.response?.status === 401 && !originalRequest.
          _retry) {
6       const { refreshToken } = loginStorePersist.getState().
          session;
7       originalRequest._retry = true;
8       const response = await axiosInstance.post("/user/
          refreshToken", {
9         refreshToken,
10      });
11      const newToken = response.data.data.accessToken;
12      loginStorePersist.getState().setLoginInfo({ data:
          response.data });
13      originalRequest.headers["Authorization"] = `Bearer ${
          newToken}`;
14      return axiosInstance(originalRequest);
15    }
16    return Promise.reject(error);
17  }
18 );
```

O trecho registra um interceptador para todas as respostas do *Axios*. A primeira função do par recebe as respostas bem-sucedidas e as devolve sem alteração. A segunda função trata os erros: ela inspeciona o código HTTP retornado e só age quando este é 401 (não autorizado) e quando a requisição original ainda não foi reexecutada, controle feito pela marcação `_retry`. Nessa situação, o *refresh token* é obtido do *store* de autenticação e enviado em uma nova requisição ao *endpoint* `/user/refreshToken`; quando essa requisição é bem-sucedida, o novo *access token* é gravado no *store* por meio de `setLoginInfo`, o cabeçalho de autorização da requisição original é atualizado e ela é reenviada de forma transparente para o usuário. Em qualquer outro caso, o erro é propagado normalmente.

4 RESULTADOS: A PLATAFORMA DE PROTOCOLOS

Neste capítulo são descritas as principais funcionalidades desenvolvidas durante o estágio para a plataforma de protocolos da empresa cliente. A apresentação está organizada por módulo, detalhando as decisões técnicas tomadas e os desafios enfrentados em cada etapa. A seleção das atividades se deu por meio da sua relevância no escopo do projeto e pela complexidade técnica envolvida.

4.1 Módulo de Autenticação e Controle de Acesso

O primeiro módulo desenvolvido foi o de autenticação, responsável por controlar o acesso dos colaboradores da empresa cliente à plataforma. A tela de login foi construída seguindo a identidade visual da empresa cliente, com campos para e-mail e senha e a opção de recuperação de senha.

A autenticação foi implementada com o uso da biblioteca *jwt-decode* para decodificação de *tokens* JWT (*JSON Web Token*) recebidos da API. Após o login bem-sucedido, os dados da sessão (*access token* e *refresh token*) e as informações do usuário extraídas do *token* eram armazenados no estado global da aplicação, gerenciado pela biblioteca *Zustand* com o *middleware* de persistência. A atualização imutável desse estado foi feita com o auxílio da biblioteca *Immer*. O Código 4.1 apresenta um trecho simplificado da função que processa o login, decodificando o *token* e populando o *store*.

Código 4.1 – Trecho do *store* de autenticação (*Zustand*) com *Immer* e *jwt-decode*.

```
export const loginStorePersist = create(
  persist((set, get) => ({
    // ...estado inicial omitido
    setLoginInfo: ({ data }) =>
      set(produce((draft) => {
        draft.isAuthenticated = true;
        draft.session = {
          accessToken: data.data.accessToken,
          refreshToken: data.data.refreshToken,
        };
        const decoded = jwtDecode(data.data.accessToken);
        draft.userData = {
          name: decoded.unique_name,
          id: decoded.nameid,
          email: decoded.email,
```

```
            roleId: decoded.role,
          };
        })),
    }), { name: "UserData" })
);
```

O *store* é criado com a função `create` do *Zustand* combinada ao *middleware* `persist`, que salva o conteúdo em armazenamento local sob o nome `UserData`, mantendo a sessão entre acessos. A função `setLoginInfo` é a operação central: ela é executada após o sucesso do login e recebe os dados retornados pela API. A chamada `produce`, do *Immer*, expõe um *draft* mutável do estado e permite escrever as alterações de forma direta. No bloco interno, o usuário é marcado como autenticado, os *tokens* são gravados na sessão, o *access token* é decodificado por `jwtDecode` e os campos relevantes (nome, identificador, e-mail e perfil) são extraídos das suas *claims* e atribuídos a `userData`.

Além da autenticação, o módulo contemplava o controle de permissões por perfil de usuário. A plataforma previa dois perfis principais, Administrador e Técnico, com diferentes níveis de acesso às funcionalidades. O perfil Administrador possuía acesso completo ao sistema, incluindo a gestão de usuários, enquanto o perfil Técnico tinha acesso restrito apenas ao registro e consulta de protocolos. O controle foi implementado no nível das rotas, por meio de um componente guardião (`ProtectedRouteGuard`) que verificava, a partir do perfil armazenado no *store*, se o usuário estava autenticado e se possuía permissão de administrador, redirecionando-o quando necessário. O Código 4.2 apresenta um trecho desse componente.

Código 4.2 – Trecho do componente de guarda de rotas por perfil de usuário.

```
const ProtectedRouteGuard = ({ children, adminRoute }) => {
  const { isAuthenticated, userData: { roleId } } =
    loginStorePersist((state) => state);
  const { isUserRole } = useRoles();

  if (!isAuthenticated) {
    return <Navigate to="/login" replace />;
  }
  if (adminRoute && !isUserRole("administrador", roleId)) {
    return <Navigate to="/" replace />;
  }
  return children;
};
```

O componente recebe como propriedades os elementos filhos (`children`) que ele encapsula e a marcação `adminRoute`, usada para indicar rotas restritas ao administrador. As primeiras linhas leem do *store* o estado de autenticação e o identificador do perfil do usuário, e obtêm do *hook* `useRoles` a função utilitária `isUserRole`. Em seguida, duas verificações são feitas: a primeira garante que o usuário esteja autenticado, redirecionando-o para `/login` quando não estiver; a segunda só se aplica às rotas marcadas como `adminRoute` e redireciona para a raiz quando o perfil do usuário não é administrador. Quando nenhuma das condições dispara redirecionamento, os filhos são renderizados normalmente.

A Figura 4.1 apresenta a tela de *login* da plataforma, que era o ponto de entrada de todos os usuários.

Figura 4.1 – Tela de *login* da plataforma.

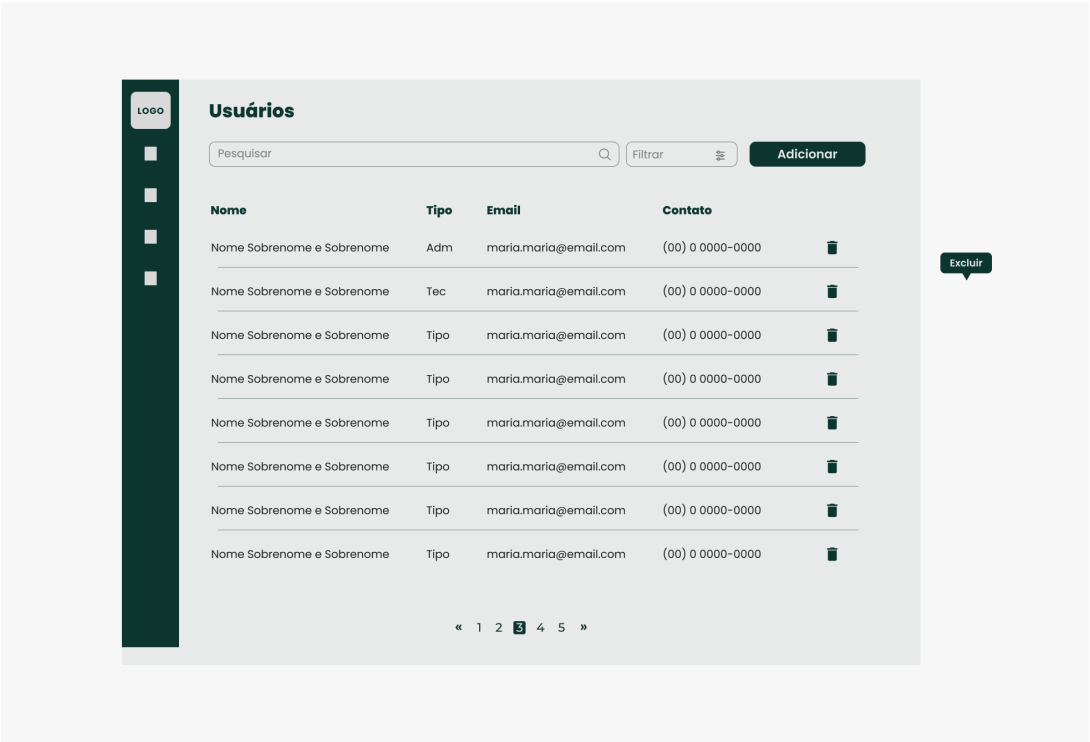


Fonte: do autor (2026).

Já a gestão de usuários, acessível apenas pelo perfil Administrador, permitia visualizar, adicionar, editar e excluir os colaboradores cadastrados. A listagem exibia nome, tipo de perfil, e-mail e contato de cada usuário, com suporte a busca textual e filtros por tipo de perfil, conforme apresentado na Figura 4.2. As operações de criação e exclusão de usuários eram realizadas por meio de modais, componentes sobrepostos à tela principal que permitem ao usuário executar uma ação sem perder o contexto da página atual. Foram implementados modais para o formulário de adição de usuário, para a confirmação de exclusão e para o *feedback* de sucesso e erro, ilustrados na Figura 4.3 junto com as variações do painel de filtros. Todos os formulários

utilizavam a biblioteca *Formik* para gerenciamento de estado dos campos e *Yup* para validação dos dados antes do envio à API.

Figura 4.2 – Tela de gestão de usuários.



Fonte: do autor (2026).

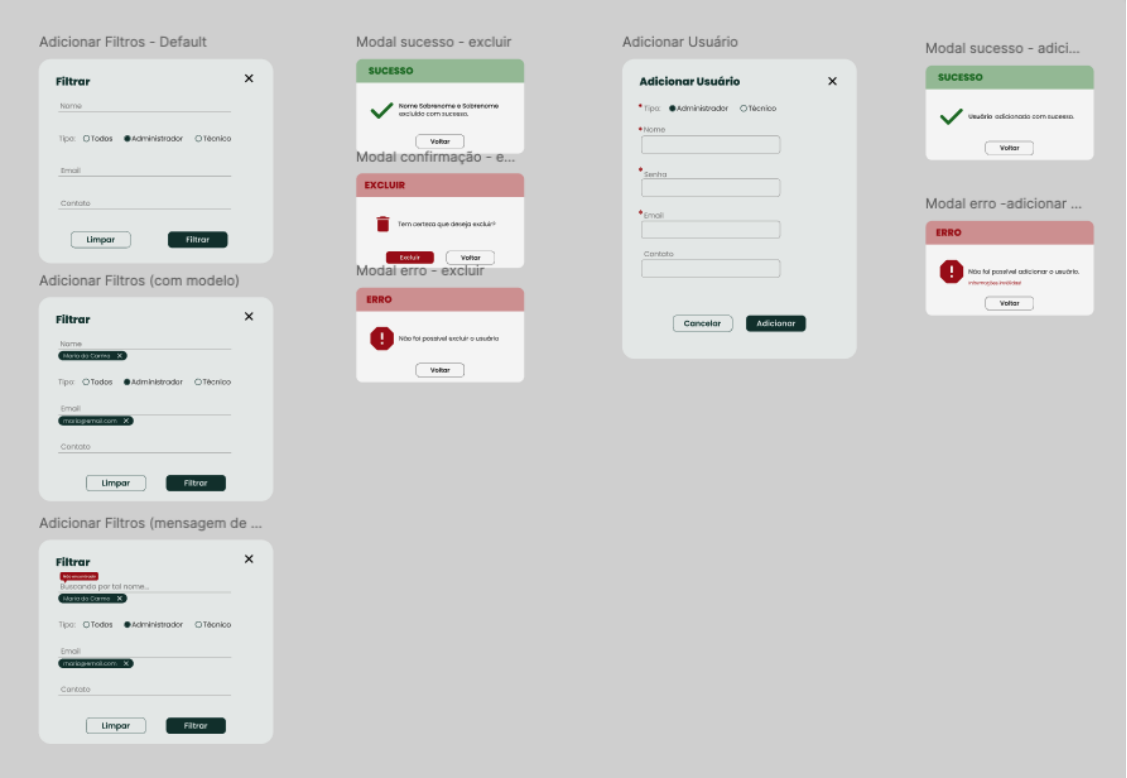
4.2 Módulo de Registros Agrícolas

O módulo de registros agrícolas constituiu o núcleo da plataforma, sendo responsável pelo cadastro, edição e consulta dos protocolos de pesquisa. Cada protocolo representava um registro completo de uma pesquisa agrícola, contendo informações sobre a safra analisada, os tratamentos aplicados, as características do solo e os resultados de produtividade.

4.2.1 Listagem de Protocolos

A tela de listagem de protocolos foi construída utilizando a biblioteca *TanStack React Table*, que oferece recursos avançados de tabela como ordenação, paginação e filtragem. Cada protocolo era exibido com seu código, tipo, cliente, classe, cultura e status atual. As ações disponíveis incluíam edição, exclusão, geração de QR Code e exportação para planilha.

Figura 4.3 – Modais de filtro, adição, exclusão e *feedback* da gestão de usuários.



Fonte: do autor (2026).

A funcionalidade de geração de QR Code foi implementada com a biblioteca *qrcode* e tinha como objetivo facilitar a identificação dos protocolos em campo. Cada QR Code gerado continha um identificador único que, ao ser escaneado, direcionava o colaborador diretamente para a página do protocolo correspondente na plataforma.

4.2.2 Formulário de Cadastro e Edição de Protocolos

O formulário de cadastro de protocolos representou um dos maiores desafios técnicos do projeto, dado o grande volume de campos e a complexidade das regras de negócio envolvidas. O formulário foi organizado em abas para melhorar a usabilidade: Informações, Avaliações, Tratamentos, Local, Solo, Aplicações, Manejo e Dashboard.

A aba de Informações continha os dados gerais do protocolo. Entre os campos obrigatórios estavam o tipo do protocolo (Ret ou Comercial), o código, o cliente, as classes do experimento, o status, o número de tratamentos e o número de repetições. Os demais campos eram opcionais, como cultura utilizada, variedade, cultura anterior, tipo de plantio, datas de plantio, colheita e emergência, adubação, espaçamento entre linhas, delineamento experimen-

tal, tamanho da parcela e parcela útil. Essa distinção entre campos obrigatórios e opcionais era expressa no esquema de validação apresentado adiante, e o campo de classes permitia a seleção de múltiplos valores, com a possibilidade de cadastrar novas classes diretamente no formulário.

O padrão visual adotado nessa aba, com os campos agrupados em duas colunas e os obrigatórios marcados com asterisco, é o mesmo apresentado na Figura 4.5, discutida adiante.

A aba de Avaliações permitia o registro dos dados coletados em cada momento de avaliação. A estrutura desta aba era dinâmica: o número de colunas da tabela variava de acordo com o número de repetições configurado na aba de Informações, e novas linhas podiam ser adicionadas conforme a necessidade. Alguns campos podiam ser definidos como fórmulas, isto é, expressões matemáticas calculadas a partir dos valores de outros campos; para avaliar essas expressões em tempo de execução, foi utilizada a função `evaluate` da biblioteca *mathjs*.

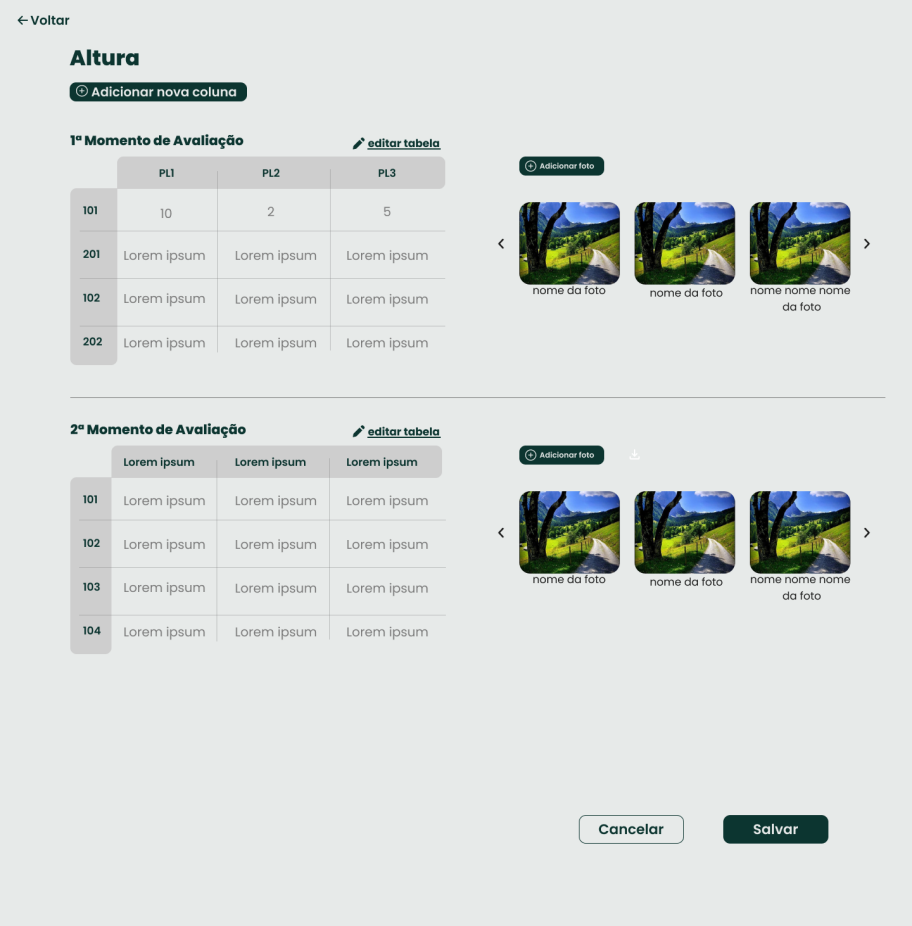
Além dos dados numéricos, a aba suportava o *upload* de fotografias de campo, permitindo que os técnicos anexassem registros visuais diretamente vinculados a cada momento de avaliação. Para facilitar o trabalho posterior, foi implementada a funcionalidade de *download* de todas as fotografias de uma avaliação em um único arquivo compactado, utilizando a biblioteca *JSZip*. As imagens eram obtidas da API por meio de uma requisição gerenciada pelo *React Query*, convertidas em *blobs*, agrupadas em um arquivo *.zip* e salvas no dispositivo do usuário, conforme o Código 4.3. A Figura 4.4 apresenta a aba de Avaliações da plataforma.

Código 4.3 – Trecho do *download* das fotografias de uma avaliação em arquivo *.zip* com *JSZip*.

```
const handleDownloadImages = async (evaluation) => {
  getImages(evaluation.id, {
    onSuccess: async ({ data }) => {
      const images = data.data;
      const blobs = await Promise.all(
        images.map((img) => fetch(img.url).then((res) => res.
          blob())))
      );
      const zip = new JSZip();
      blobs.forEach((blob, i) => zip.file(`${images[i].title}.
        jpg`, blob));
      const zipFile = await zip.generateAsync({ type: "blob" })
        ;
      saveFile(zipFile, evaluation.name);
    },
  });
};
```

A função recebe a avaliação selecionada e dispara a busca das imagens pela API por meio do *mutation* `getImages`, fornecido pelo *React Query*. No *callback* de sucesso, a lista de imagens retornada é percorrida e cada item é mapeado para uma promessa que baixa o arquivo como *blob*, utilizando o `fetch` nativo do navegador; as promessas são aguardadas em paralelo com `Promise.all`. Em seguida, uma instância de `JSZip` é criada e cada *blob* é adicionado ao arquivo com o nome correspondente. Por fim, o método `generateAsync` produz o arquivo `.zip` final em memória, que é entregue ao usuário pela função utilitária `saveFile`.

Figura 4.4 – Aba de Avaliações do formulário de protocolo.

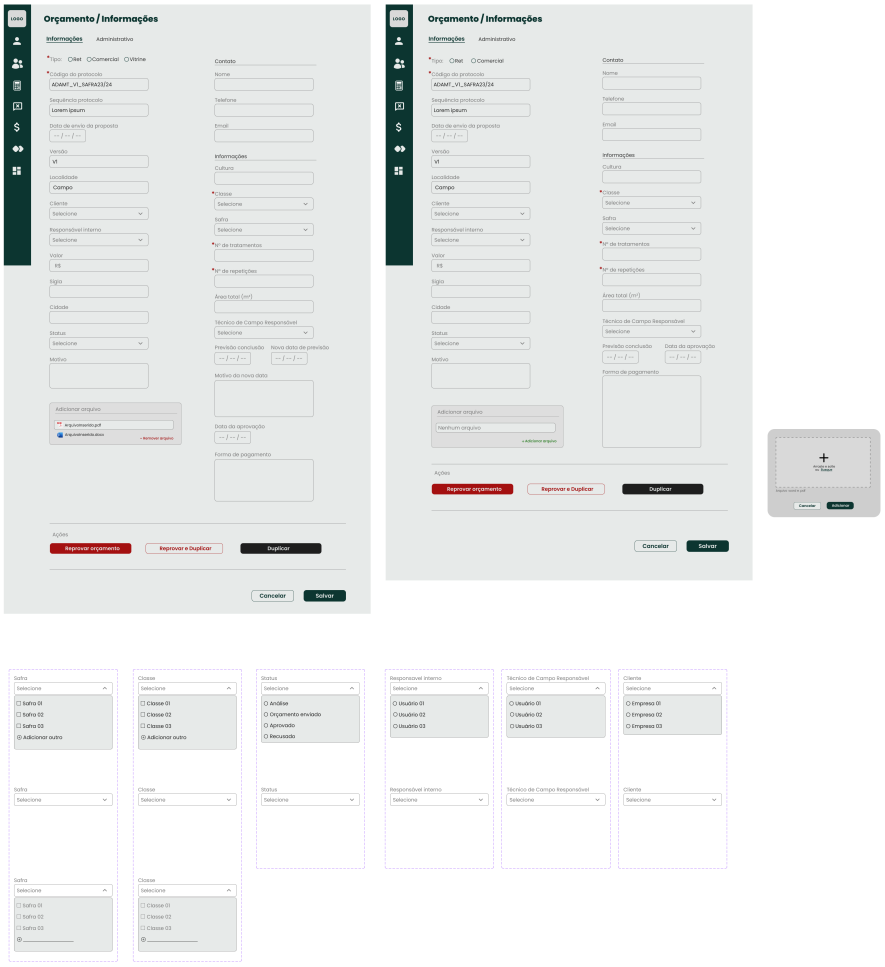


Fonte: do autor (2026).

O gerenciamento do estado do formulário foi implementado com a biblioteca *Formik*, encapsulada em um *hook* customizado (`useForm`) que padronizava o controle dos valores dos campos e a orquestração das validações em todas as telas. Essa padronização era importante, dado que um protocolo podia conter dezenas de tratamentos, cada um com múltiplas repetições e avaliações, resultando em estruturas de dados profundamente aninhadas.

O mesmo padrão de organização de formulários (estrutura em abas, agrupamento de campos por área e separação visual entre seções obrigatórias e opcionais) foi reutilizado em outros módulos da plataforma, como o de orçamentos. A Figura 4.5 apresenta variações da tela de Informações desse módulo, incluindo o estado padrão e estados com mensagens de validação, além de uma amostra dos componentes de seleção (*selects*) reaproveitados em diferentes contextos.

Figura 4.5 – Tela de Informações do módulo de orçamentos e componentes reutilizados.



Fonte: do autor (2026).

Uma das principais dificuldades encontradas neste módulo foi a tradução das regras de negócio da empresa cliente para a lógica do formulário. As regras variavam conforme o tipo de protocolo, a cultura selecionada e a classe do experimento, exigindo uma comunicação constante com a equipe do cliente para validar o comportamento esperado. O uso da biblioteca *Yup* para a definição de esquemas de validação mostrou-se essencial para garantir a integridade dos dados antes do envio ao servidor, distinguindo os campos obrigatórios dos opcionais e

validando tipos, listas e formatos. O Código 4.4 apresenta um trecho do esquema de validação da aba de Informações do protocolo.

Código 4.4 – Trecho do esquema de validação (*Yup*) da aba de Informações do protocolo.

```
const validationSchema = Yup.object().shape({
  isRET: Yup.bool().required("O campo e obrigatorio"),
  code: Yup.string().required("O campo e obrigatorio"),
  culture: Yup.string(),
  experimentalDesign: Yup.string().nullable(),
  numberOfTreatments: Yup.number().required("O campo e
     obrigatorio").nullable(),
  numberOfRepeats: Yup.number().required("O campo e obrigatorio
     ").nullable(),
  plantingDate: Yup.string().nullable(),
  harvestDate: Yup.string().nullable(),
  classes: Yup.array().min(1, "O campo e obrigatorio"),
  clientId: Yup.string().required("O campo e obrigatorio"),
});
```

O esquema é declarado com `Yup.object().shape`, recebendo um objeto cujas chaves correspondem aos campos do formulário. Cada chave aciona uma cadeia de validações específica do seu tipo: os campos do tipo *string* ou *bool* usam `.required` quando são obrigatórios; os campos numéricos combinam `.required` com `.nullable`, o que permite manter o campo nulo durante o preenchimento parcial, mas exige o seu preenchimento no envio; o campo `classes` utiliza `array().min(1)` para garantir ao menos uma classe selecionada; e os campos puramente opcionais (como `culture`) aparecem sem `.required`. A mensagem passada a cada validação é o texto exibido ao usuário quando a regra é violada.

4.3 Módulo de Análise e Dashboards

O módulo de análise e *dashboards* representou o principal diferencial da plataforma em relação ao uso de planilhas. A proposta era transformar os dados brutos registrados nos protocolos em visualizações gráficas que auxiliassem os pesquisadores na tomada de decisões.

4.3.1 Geração Automatizada de Gráficos

A geração de gráficos foi implementada utilizando a biblioteca *ApexCharts*, integrada ao React por meio do pacote *react-apexcharts*. Os componentes de gráfico recebiam os dados

já consolidados (rótulos do eixo horizontal e valores correspondentes) e montavam, de forma automática, as séries e as opções de configuração do *ApexCharts*, sem a necessidade de ajuste manual por parte do usuário. Foram utilizados principalmente gráficos de barras verticais e horizontais para comparar os resultados entre diferentes tratamentos e categorias. Cada gráfico era renderizado individualmente em um *card*, com a opção de *download* individual.

O processo de transformação dos dados em visualizações seguia um fluxo bem definido, desde a obtenção dos dados até a exportação da imagem final, conforme apresentado na Figura 4.6.

Figura 4.6 – Fluxo de geração de gráficos a partir dos dados de avaliação.



Fonte: do autor (2026).

O ponto central desse fluxo era a montagem das séries e das opções do gráfico a partir dos vetores de rótulos (`xAxis`) e valores (`yAxis`) recebidos pelo componente. O Código 4.5 apresenta um trecho dessa configuração para um gráfico de colunas.

Código 4.5 – Trecho da configuração de séries e opções de um gráfico com *react-apexcharts*.

```
const series = [{ name: xAxisTitle, data: yAxis }];

const options: ApexOptions = {
  chart: { type: "bar", toolbar: { show: false } },
  plotOptions: { bar: { distributed: true } },
  dataLabels: { enabled: true },
  xaxis: { categories: xAxis },
  yaxis: { max: maxValue + 1 },
};

return <ApexCharts options={options} series={series} type="bar"
    />;
```

O vetor `series` contém uma única série de dados, com o rótulo informado pela propriedade `xAxisTitle` e o conjunto de valores em `yAxis`. O objeto `options`, tipado por `ApexOptions`, agrupa as configurações visuais do gráfico: `chart` define o tipo `bar` e oculta a barra de ferramentas padrão; `plotOptions` ativa a coloração distribuída por categoria; `dataLabels` habilita a exibição dos valores numéricos sobre as colunas; `xaxis` informa as categorias do eixo hori-

zontal a partir do vetor `xAxis`; e `yaxis` ajusta o valor máximo do eixo vertical com base no valor máximo dos dados. A última linha entrega esses dois objetos ao componente `ApexCharts`, que cuida da renderização propriamente dita.

Uma funcionalidade importante foi a exportação dos gráficos em formato PNG com a identidade visual da empresa cliente, contendo o logotipo da empresa e o título dos dados. Para isso, foi mantido um elemento oculto na tela contendo o gráfico acompanhado do logotipo, e a sua captura como imagem foi realizada com a biblioteca *use-react-screenshot*. A imagem resultante era então disponibilizada para *download* por meio da criação dinâmica de um *link*, conforme o Código 4.6.

Código 4.6 – Trecho da exportação de um gráfico em PNG com *use-react-screenshot*.

```
const [_, takeScreenshot] = useScreenshot();

const downloadChart = async (format) => {
  const imageBase64 = await takeScreenshot(imageRef.current);
  const link = document.createElement("a");
  link.href = imageBase64;
  link.download = `${imageTitle} - ${chartTitle}.${format}`;
  link.click();
};
```

A primeira linha obtém, do *hook* `useScreenshot`, a função `takeScreenshot`, usada para gerar uma imagem a partir de um elemento do DOM. A função `downloadChart` é assíncrona e recebe como argumento o formato de imagem desejado. No seu corpo, `takeScreenshot` captura o elemento referenciado por `imageRef`, que contém o gráfico acompanhado do logotipo, devolvendo a imagem em *base64*. Em seguida, um elemento de *link* é criado dinamicamente, o atributo `href` recebe a imagem capturada, o atributo `download` define o nome do arquivo final (combinando o título da imagem, o título do gráfico e o formato) e o clique programático no *link* força o navegador a salvar a imagem no dispositivo do usuário.

A Figura 4.7 apresenta o resultado da geração automatizada dos gráficos de barras a partir dos dados de uma avaliação. Cada gráfico ocupa um *card* próprio, dispostos em uma grade na tela, com as colunas coloridas por categoria e os valores numéricos exibidos sobre elas, de modo que o pesquisador consegue comparar os momentos de avaliação sem precisar montar manualmente as visualizações. Além do *download* individual disponível em cada *card*, a tela oferecia a opção de baixar todos os gráficos de uma só vez.

Figura 4.7 – Gráficos de barras gerados automaticamente para uma avaliação.



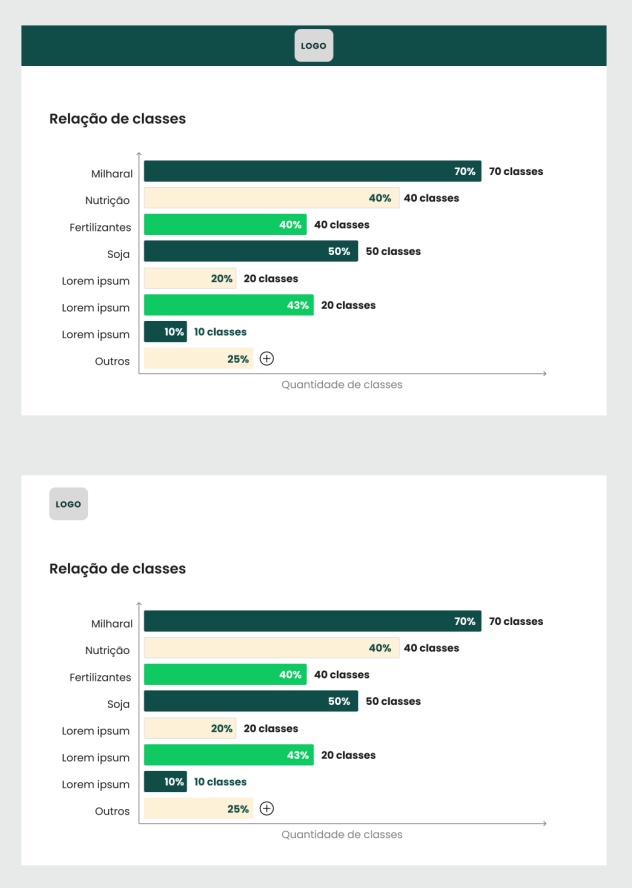
Fonte: do autor (2026).

Já a Figura 4.8 ilustra o produto final do processo de exportação descrito no Código 4.6. A imagem capturada reúne, em um único arquivo PNG, o gráfico selecionado, o logotipo da empresa cliente e o título dos dados representados, resultando em um material pronto para ser incluído nos relatórios entregues pela empresa aos seus próprios clientes.

4.3.2 Dashboard Geral

Além dos gráficos individuais por protocolo, a plataforma contava com um *dashboard* geral que apresentava uma visão consolidada de todos os protocolos cadastrados, incluindo a distribuição de protocolos por classe, por cultura e por status.

Figura 4.8 – Imagem PNG exportada de um gráfico, com a identidade visual da empresa.



Fonte: do autor (2026).

A Figura 4.9 apresenta essa tela, em que os gráficos de pizza resumem a divisão dos protocolos por tipo e por status e os gráficos de barras horizontais detalham a distribuição por classe, por cultura e por cliente.

Os dados consolidados desse dashboard, como a contagem e o percentual de protocolos por cultura, por classe e por cliente, eram fornecidos por um *endpoint* específico do *back-end* e buscados via API utilizando o *React Query* (*TanStack Query*), que gerenciava o cache das requisições e garantia que as visualizações estivessem sempre atualizadas sem a necessidade de recarregamentos manuais da página. No *front-end*, esses dados eram mapeados para os vetores de rótulos e valores consumidos pelos componentes de gráfico.

Uma das dificuldades técnicas enfrentadas na construção dos *dashboards* foi a performance de renderização quando o volume de dados era elevado. A renderização simultânea de múltiplos gráficos com grandes conjuntos de dados causava lentidão perceptível na interface. Para mitigar esse problema, foi adotada uma estratégia de carregamento sob demanda (*lazy lo-*

Figura 4.9 – *Dashboard* geral com a visão consolidada dos protocolos.



Fonte: do autor (2026).

ading), em que os gráficos eram renderizados apenas quando o usuário navegava até a seção correspondente.

5 CONSIDERAÇÕES FINAIS

Este capítulo apresenta uma avaliação do produto desenvolvido durante o estágio e uma reflexão sobre o impacto da experiência na formação acadêmica e profissional do autor. Retomam-se, de forma sintética, os objetivos definidos na introdução, discutindo em que medida foram alcançados e quais limitações e aprendizados foram observados ao longo do processo.

5.1 Avaliação do Produto Desenvolvido

O objetivo central do projeto era substituir o registro descentralizado de dados de pesquisa, antes mantido em planilhas eletrônicas, por uma plataforma web própria capaz de centralizar os protocolos da empresa cliente, controlar o acesso por perfil de usuário e gerar análises visuais de forma automatizada. Diante das funcionalidades entregues, considera-se que esse objetivo foi atendido no plano técnico.

A arquitetura adotada para o *front-end*, organizada em camadas de apresentação, estado e acesso a dados, mostrou-se adequada para acomodar o volume de telas e a complexidade do domínio. A camada de autenticação e o controle de acesso garantiram que cada colaborador interagisse apenas com as funcionalidades pertinentes ao seu papel, com restrição feita no nível das rotas e com renovação transparente de sessão por meio do mecanismo de *refresh token*. O módulo de registros agrícolas implementou o cadastro completo dos protocolos, incluindo formulários extensos com validação declarativa por esquemas e estruturas dinâmicas para as avaliações de campo, com suporte a campos calculados por fórmulas e ao agrupamento de fotografias em arquivos compactados. Por fim, o módulo de análise concretizou o principal diferencial da plataforma em relação às planilhas: a geração automatizada de gráficos e *dashboards*, alimentados por dados consolidados pelo *back-end* e exportáveis como imagens padronizadas pela identidade visual da empresa.

Entre as principais dificuldades superadas, destacam-se a tradução das regras de negócio do domínio agronômico para a lógica dos formulários, que exigiu validação contínua junto ao cliente, e a manutenção de um desempenho adequado na renderização simultânea de múltiplos gráficos, mitigada pela adoção de carregamento sob demanda. Como limitação, observou-se que a adoção da plataforma pelo cliente foi inicialmente reduzida, o que pode ser atribuído

à mudança de cultura exigida pela transição das planilhas para um sistema estruturado, e não a uma deficiência técnica da solução. Trabalhos futuros poderiam incluir o acompanhamento dessa adoção ao longo do tempo, a realização de testes de usabilidade com os colaboradores e a evolução dos *dashboards* com novos tipos de análise.

5.2 Impacto do Estágio na Formação Acadêmica e Profissional

O estágio representou a primeira experiência profissional do autor com o desenvolvimento de *software* em um produto real, em equipe estruturada e com um cliente externo. Essa vivência permitiu aplicar, em um contexto concreto, conhecimentos adquiridos ao longo da graduação em disciplinas de engenharia de *software*, desenvolvimento web, banco de dados e interação humano-computador, evidenciando como esses conteúdos se articulam na prática.

No plano técnico, a experiência consolidou competências em desenvolvimento *front-end* com React e TypeScript, no gerenciamento de estado global com Zustand e Immer, na construção de formulários com Formik e Yup, na integração com APIs por meio de Axios e React Query e, sobretudo, na visualização de dados com ApexCharts, área que passou a orientar os interesses profissionais do autor. A adoção da metodologia Scrum proporcionou contato com o trabalho iterativo e incremental, com reuniões de planejamento e acompanhamento, e com a importância das entregas frequentes ao cliente.

No plano interpessoal, o estágio desenvolveu habilidades de comunicação e de trabalho em equipe, especialmente no levantamento de requisitos e na tradução de necessidades de negócio em soluções técnicas, atividade que exigiu diálogo constante com o cliente e com os demais membros da equipe. Em uma dimensão acadêmica, a redação deste relatório consolidou a prática de documentar decisões de projeto, explicitar trade-offs e refletir criticamente sobre o trabalho desenvolvido, habilidades igualmente importantes para a atuação profissional.

Cabe ainda destacar o caminho inverso: o impacto que a formação acadêmica teve sobre o estágio. As disciplinas que mais contribuíram diretamente para o trabalho desenvolvido foram Engenharia de Software, que forneceu a base para compreender o ciclo de desenvolvimento, o levantamento de requisitos e a aplicação de metodologias ágeis; Programação Orientada a Objetos, cujos conceitos de modularidade e responsabilidade única se traduziram naturalmente na componentização do React; Banco de Dados, que facilitou o entendimento das estruturas retornadas pela API e o desenho dos formulários que produziam e consumiam esses dados; e

Interação Humano-Computador, que orientou as decisões de usabilidade e de organização das informações em tela. Disciplinas de Algoritmos e Estruturas de Dados, embora menos visíveis no dia a dia, sustentaram a manipulação dos vetores e estruturas aninhadas que apareceram em formulários, tabelas e gráficos.

Por outro lado, foram percebidas lacunas que, se tivessem sido cobertas durante a graduação, teriam reduzido a curva de aprendizado inicial. A primeira é o contato com *frameworks* modernos de *front-end*, como o React e o ecossistema em torno do TypeScript, normalmente apresentados de forma superficial nas disciplinas obrigatórias. A segunda é a prática de testes automatizados de interface, pouco trabalhada ao longo do curso e cada vez mais valorizada no mercado. Por fim, uma maior exposição a práticas de integração contínua, de versionamento em equipe (com *pull requests* e revisões de código) e de *deploy* de aplicações em ambientes de nuvem teria facilitado a inserção no fluxo de trabalho da empresa desde os primeiros dias do estágio. Nesse mesmo grupo entram noções de conteinerização e de infraestrutura, como o uso de *Docker* e a configuração de ambientes de execução, presentes tanto neste projeto quanto em outros acompanhados ao longo do estágio.

De modo geral, a experiência contribuiu de forma significativa para o amadurecimento profissional e para a definição de um direcionamento de carreira voltado ao desenvolvimento de interfaces orientadas a dados, integrando a construção de aplicações web à análise e visualização de informações.

REFERÊNCIAS

Centro de Estudos Avançados em Economia Aplicada; Confederação da Agricultura e Pecuária do Brasil. **PIB do Agronegócio Brasileiro**. 2025. Série histórica e boletins técnicos. Disponível em: https://www.cepea.esalq.usp.br/br/pib-do-agronegocio-brasileiro.aspx. Acesso em: 07 mar. 2026.

CHHIPA, J. **ApexCharts: Modern & Interactive Open-source Charts**. 2024. Documentação oficial. Disponível em: https://apexcharts.com/. Acesso em: 10 mar. 2026.

Empresa Brasileira de Pesquisa Agropecuária. **Visão 2030: o futuro da agricultura brasileira**. Brasília: Embrapa, 2019. Disponível em: https://www.embrapa.br/visao/o-futuro-da-agricultura-brasileira. Acesso em: 07 mar. 2026.

KATO, D. **Zustand: Bear necessities for state management in React**. 2024. Repositório GitHub. Disponível em: https://github.com/pmndrs/zustand. Acesso em: 10 mar. 2026.

LINSLEY, T. **TanStack Query: Powerful asynchronous state management**. 2024. Documentação oficial. Disponível em: https://tanstack.com/query/latest. Acesso em: 10 mar. 2026.

MASSRUHá, S. M. F. S.; LEITE, M. A. de A. *et al.* **Agricultura Digital: pesquisa, desenvolvimento e inovação nas cadeias produtivas**. Brasília: Embrapa, 2020.

Meta Platforms. **React: The library for web and native user interfaces**. 2024. Documentação oficial. Disponível em: https://react.dev/. Acesso em: 10 mar. 2026.

Microsoft. **TypeScript: JavaScript With Syntax For Types**. 2024. Documentação oficial. Disponível em: https://www.typescriptlang.org/. Acesso em: 10 mar. 2026.

MUI. **Material UI: React components that implement Google's Material Design**. 2024. Documentação oficial. Disponível em: https://mui.com/. Acesso em: 10 mar. 2026.

PALMER, J. **Formik: Build forms in React, without the tears**. 2024. Documentação oficial. Disponível em: https://formik.org/. Acesso em: 10 mar. 2026.

SCHWABER, K.; SUTHERLAND, J. **The Scrum Guide: The Definitive Guide to Scrum: The Rules of the Game**. Scrum.org, 2020. Disponível em: https://scrumguides.org/scrum-guide.html. Acesso em: 10 mar. 2026.

Zeeway Soluções Digitais. **Institucional Zeeway**. 2026. Site institucional. Disponível em: https://zeeway.com.br/. Acesso em: 07 mar. 2026.